# DataFlash: A Secure, Intent-Driven Visual Workflow Framework for Autonomous Data Mining and Analytics

Minyang Han
*High School Student*
*Shanghai Jinyuan Senior High School*
Shanghai, China
hanqic@hotmail.com

Zhiping Xu
*Fudan University*
Shanghai, China
zpxu@fudan.edu.cn

***Abstract*—Visual workflow tools organize data analysis as connected operations, while large language models (LLMs) can generate transformations from natural-language instructions. DataFlash combines these capabilities in a local-first prototype. Built-in operators, user extensions, and intent-driven AI nodes share a directed acyclic graph, typed ports, and an artifact cache. Generated Python is checked by a default-deny Abstract Syntax Tree (AST) policy and executed in a spawned worker with restricted builtins, selected socket API patches, and resource controls. These application-level controls address selected unsafe operations; they do not provide complete filesystem or network confinement. Saved workflows and locked code support re-execution under unchanged inputs and dependencies. A passenger-survival case study illustrates the execution path. Existing prototype traces report 58–70 ms of local validation and worker overhead per AI node, excluding model calls. The evaluation demonstrates prototype feasibility; it does not establish generation correctness, repair effectiveness, or a causal productivity improvement.**

***Index Terms*—Data Mining Systems, Visual Analytics, Large Language Models, AST Sandbox, Low-Code Workflows.**

## I. Introduction

### A. Background

Orange3 [1] and KNIME [2] let analysts connect data loading, filtering, aggregation, and visualization operations. Tasks outside the available operators require extensions or scripts. LLMs offer another way to specify these transformations: a user provides an instruction and the model generates code [3], [4]. However, generated code may access files, invoke external processes, or perform unintended operations. Repeated generation may also produce different programs. DataFlash integrates code generation into visual nodes and separates code admission, execution, and reuse.

### B. Problem Statement

We investigate how a visual workflow can accept generated transformations, check selected unsafe operations, and preserve the accepted program for later runs. Prompt instructions alone cannot enforce execution restrictions.

### C. Contributions

1) A common node interface for built-in operations, user extensions, and LLM-generated transformations, with typed ports and DAG scheduling.
2) An AST admission policy followed by a spawned worker with restricted builtins and resource controls. The design provides application-level defense in depth, rather than a complete OS sandbox.
3) Workflow serialization, code locking, and recursive cache fingerprints for artifact reuse, demonstrated through a single exploratory case study.

## II. System Architecture and Design

DataFlash follows a four-layer local-first design (Fig. 1): a browser client, a Flask server, an execution engine, and an isolation boundary. The client and server run locally. Model requests send the intent and schema metadata to a remote provider; optional masked samples may also be transmitted. The provider receives the API credential for authentication. Local storage therefore does not imply that all information remains on the host.

### A. Visual Engine and DAG Execution

The front end is a React Flow canvas on which nodes are dragged, connected via typed ports, and edited through an inspector. A workflow is a graph $G = (V, E)$ where each vertex $v \in V$ carries a type, parameters, and—for dynamic nodes—an optional cached code body, and each edge $e = (u, v) \in E$ binds an output port of $u$ to an input port of $v$.

Execution begins by validating that $G$ is acyclic and topologically ordering it with Kahn's algorithm. Letting $\text{indeg}(v)$ be the in-degree of $v$, the engine repeatedly emits vertices of in-degree zero and decrements their successors; if fewer than $|V|$ vertices are emitted, the residual set $\{v : \text{indeg}(v) > 0\}$ is reported to the user as the offending cycle. The linear order is then partitioned into *layers* $L_1, L_2, \ldots, L_k$, where

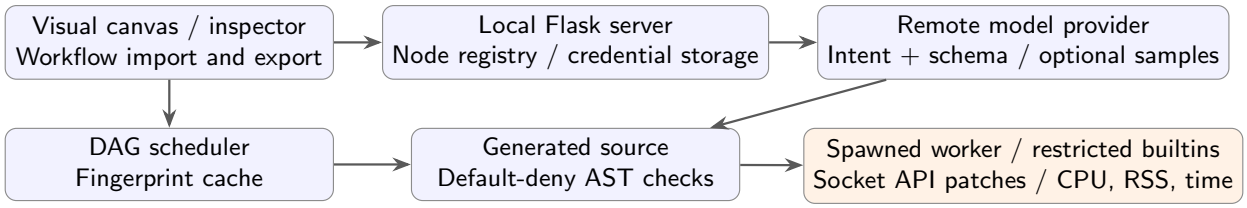



Fig. 1. DataFlash execution path. Generated code passes AST checks before execution in a spawned worker. Socket API patches and resource controls are defense-in-depth measures; the worker does not enforce OS-level filesystem or network confinement. Cached artifacts support subsequent runs.

$$U_{i-1} = \bigcup_{j<i} L_j,$$
$$L_i = \big\{ v \in V \setminus U_{i-1} \;\big|\; \mathrm{preds}(v) \subseteq U_{i-1} \big\}.$$

so that all vertices within a layer are mutually independent and may execute concurrently. Deterministic nodes in a layer run on a bounded thread pool; AI nodes are serialized within their layer in the current prototype.

Node outputs are materialized to disk as Parquet (tables), JSON (figures and statistics), or Markdown (reports). Large tables are passed *by reference*: the engine hands downstream nodes a file path rather than an in-memory DataFrame, which avoids copying tables through the scheduler and matches the path-only contract of the isolation boundary (Sec. II-C).

### B. Intent-Driven AI Engine

A node is marked dynamic by the flag `is_ai: true`. When such a node executes without a locked code body, the engine (i) constructs a schema summary of the upstream table—column names, dtypes, row count, and optionally *masked* samples—(ii) composes a system+user prompt embedding that summary and the user's natural-language `intent`, and (iii) calls an OpenAI-compatible endpoint (ZenMux) requesting a strict JSON object. A contract parser enforces that the response contains a non-empty `code` string (plus `explanation` and `assumptions`); fenced code blocks are unwrapped and malformed responses trigger up to two contract-repair retries before the call is abandoned.

Generated code obeys a fixed calling convention: the upstream table is bound to `df`, and the fragment must assign its result to `result` (for tables and statistics) or `fig` (for Plotly figures). Schema metadata supplies column names and types but does not guarantee correct column use or correct analysis. Masking samples is not a formal privacy guarantee.

### C. Security Model: The AST Allowlist

**Threat model and scope.** We assume a trusted local server, Python interpreter, and installed libraries. Generated code and uploaded extensions are untrusted inputs. The controls target accidental side effects and common injection patterns involving imports, reflection, file access, and selected process/network APIs. Deliberate escapes through native libraries, interpreter or kernel vulnerabilities, and compromised dependencies are outside the protection claimed here. The worker inherits host permissions; it is not an OS security boundary. In the title, "secure" refers to these scoped controls, not arbitrary-code confinement.

The static control parses source with a default-deny AST allowlist. The generated source is parsed with Python's `ast` module and walked; a node is admitted only if its syntactic type belongs to an explicit allowlist $\mathcal{N}$ of expressions, assignments, conditionals, loops, comprehensions, and function definitions. Formally, code $c$ is accepted iff

$$\forall\, n \in \mathrm{AST}(c) :\ \mathrm{type}(n) \in \mathcal{N} \ \wedge\ \Phi(n),$$

where $\Phi(n)$ is a conjunction of targeted predicates. Constructs absent from $\mathcal{N}$—`import`, `try`/`except`, `with`, `class`, `global`, `del`, `yield`, `await`, the walrus operator, and `match`—are rejected outright, each with a localized, human-readable reason surfaced in the UI. The predicate $\Phi$ additionally blocks:

- *Dunder access*: any name, attribute, or string subscript beginning with __, which rejects direct use of the `().__class__.__bases__` sandbox-escape family.
- *Forbidden calls*: `eval`, `exec`, `compile`, `open`, `getattr`/`setattr`, `__import__`, `input`, and related reflective builtins.
- *Forbidden attributes*: escape hatches inside otherwise-allowed libraries—`read_pickle`/`to_pickle` (arbitrary code on load), `eval`/`query` (string compilation), `system`, `popen`, `Popen`, `urlopen`, `socket`, `remove`, `rmtree`, and `ctypes`.
- *Protected rebinding*: assignment to `__builtins__` or decorator use on function definitions.

The validator runs under two *profiles*. The base profile governs static user extensions; the stricter *AI profile* additionally forbids every pandas file-I/O method (`read_csv`, `to_parquet`, `read_json`, ...) to reject known file-I/O entry points; this is not proof that all library paths to the filesystem are blocked. In-memory serializers such as

`to_dict` and `to_markdown` remain available. Redundant imports of modules the sandbox already injects (`pandas`, `numpy`, `plotly`, `json`, ...) are silently stripped rather than rejected, since models emit them habitually and an import such as `import os` still fails validation.

Static admission is only the first tier. Admitted code is executed in a separate worker process described in Sec. III-A. The allowlist is a *static gatekeeper*, not a proof that every Python escape is syntactically recognizable: Python reflection and library behavior make such completeness a separate formal-verification problem. Consequently, safety claims rely on both tiers and are limited to the threat model and controls stated below.

## III. Implementation Details

### A. Backend and Process Isolation

The backend is a Flask 3 application on Python 3.12 exposing a REST + Server-Sent-Events API; run progress (`node_start`, `node_done`, `node_error`, `run_done`) is streamed to the client so the canvas animates as the DAG executes. Validated code does *not* run in the server process. Instead the engine uses a `spawn` multiprocessing context to launch a fresh worker whose controls are applied before any user code runs:

- **Restricted builtins**: the execution namespace receives a curated `SAFE_BUILTINS` map (numeric, container, and iterator primitives) instead of the full builtin table; `pandas`, `numpy`, and `plotly` are pre-injected so no `import` is needed.
- **Selected socket APIs patched**: `socket.socket` and `socket.create_connection` are monkey-patched to raise, so even code admitted by the validator cannot use those standard socket entry points.
- **Resource limits**: `RLIMIT_CPU` caps CPU seconds; a parent-side RSS watchdog kills the worker if resident memory exceeds a threshold (macOS ignores `RLIMIT_AS`, so the watchdog supplies the RSS check); a wall-clock timeout terminates runaway processes.
- **Path-only IPC**: the parent sends the worker input file paths and a target output path and receives back only a status record rather than a DataFrame over the pipe. This reduces object transfer; it does not authenticate worker-written artifacts.

These controls limit CPU and wall-clock execution and terminate workers on observed RSS excess; a sampled watchdog may allow transient memory overshoot. It is a lightweight process boundary, not a VM, container, or formally verified filesystem/network sandbox; the socket patch and AST policy therefore remain defense-in-depth controls rather than a claim of complete Python confinement.

### B. Fingerprint Cache and Self-Healing

Each node is assigned a content fingerprint

$$f(v) = H\big(\text{type}, \text{params}, \text{code}, s(v), \{f(u) : (u, v) \in E\}\big),$$

a hash over the node's type, parameters, code body, an input file-stamp $s(v)$ for source nodes, and the fingerprints of all upstream nodes. Because the fingerprint is recursive, a change anywhere upstream invalidates exactly the affected sub-DAG and no more; if every output port of $v$ is already present in the cache under $f(v)$, the node is skipped and its artifact reused. Cache hits avoid executing unchanged nodes; interactive response time is not measured here.

A bounded repair loop handles validation and runtime failures. If an AI node's generated code fails validation or raises at run time, the traceback and the failing source are fed back to the model with a repair instruction, up to two additional attempts. Each repaired fragment repeats validation and worker execution. The loop can attempt to correct a wrong column name or dtype mismatch, but successful execution does not establish analytical correctness. Repair success was not measured. It does not make generation deterministic: DataFlash records and locks accepted code so that subsequent execution reuses the same artifact, but changing the model, prompt, or decoding settings may produce a different first artifact.

### C. Dynamic Node Expansion

Third-party nodes are shipped as directories containing a `manifest.json` and, for deterministic nodes, a `transform.py`. The manifest declares the node's identity, typed ports, and a parameter schema drawn from an allowlisted set of UI controls:

```
{ "id": "scale_numeric", "result_kind": "table",
  "inputs": [{"id":"table_in","type":"table"}],
  "outputs": [{"id":"table_out","type":"table"}],
  "params_schema": [
    {"key":"columns","type":"columns","required":true},
    {"key":"factor","type":"number","default":1.0}]}
```

On upload, the loader validates the identifier (`^[a-z][a-z0-9_]{1,47}$`), parses and type-checks every port and parameter, and—critically—runs `transform.py` through the same AST validator used for LLM code before the node is ever registered. Custom nodes are namespaced under `custom.<id>` and may not shadow a built-in. Setting `is_ai: true` makes `transform.py` optional: the manifest is augmented with mandatory `intent` and `model` parameters, and the node's behavior is synthesized at run time exactly as in Sec. II-B. The lifecycle—*validate manifest → type-check ports → AST-check code → register namespaced spec*—is identical for static and AI packages, so a single extension mechanism spans both.

### D. Local Credential Storage

The provider API key is encrypted at rest with Fernet (AES-128-CBC + HMAC): a per-installation key file is created with `0600` permissions on first use, and the API key is stored only as ciphertext. The key is never returned to the browser in plaintext and never committed to a workflow file, so a shared `.flow.json` carries the *structure* of an analysis but no provider credential. Prompts and code may still contain sensitive text, and the encryption key is on the same host;

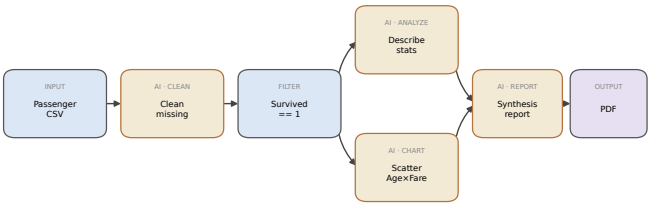



Fig. 2. The reference case-study DAG. Deterministic nodes (blue) and intent-driven AI nodes (amber) share one wiring and caching semantics; the Analyze and Chart branches share a layer but AI execution is serialized.

encryption at rest does not protect against a compromised local account.

## IV. Evaluation and Case Study

### A. Reference Pipeline

We use a descriptive passenger-survival workflow as the running case study (Fig. 2). A **File Input** node loads a CSV; an **AI Clean** node is given the intent *"drop rows where Age is null; strip and lowercase Sex and Embarked"*; a deterministic **Filter** keeps survivors (`Survived == 1`); the graph then fans out to an **AI Analyze** node (descriptive statistics) and an **AI Chart** node (an Age×Fare scatter), both feeding an **AI Report** node whose Markdown synthesis is rendered to **PDF**. The graph exercises built-in filtering, generated transformations, fan-out scheduling, worker execution, and report export. It does not train or evaluate a survival predictor.

**Available data and preprocessing.** The distributed node examples include a local `passengers.csv` demonstration table with 30 rows and six fields: `Survived`, `Pclass`, `Sex`, `Age`, `Fare`, and `Embarked`. It is a passenger-survival example; its upstream provenance is not documented, so we do not identify it as the full Titanic benchmark. `Survived` is the binary outcome field; the remaining five fields describe passengers. One `Age` value is missing and the other fields are complete. The stated cleaning rule removes that row, leaving 29, and lowercases and strips the two text fields without imputation. Filtering for survivors leaves 16 rows. These counts describe the available example, not a verified reconstruction of the original timing input: the prototype traces did not preserve a dataset identifier.

### B. Exploratory Evaluation Setup

We instrumented four authoring paths for the same analysis and report: writing a complete Python/pandas script from a blank file, assembling Orange3 and KNIME workflows from their widget libraries, and expressing each transformation as an intent in DataFlash. These are intentionally different interaction regimes: the Raw Python path measures unaided from-scratch scripting, whereas DataFlash includes LLM-assisted intent compilation. The comparison therefore illustrates end-to-end authoring modes; it does *not* isolate the benefit of the visual interface from that of LLM assistance. A fair attribution study should add an LLM-assisted Python baseline (e.g., the same model in a chat or IDE assistant) and stratify participants by prior Python, data-analysis, and visual-tool experience.

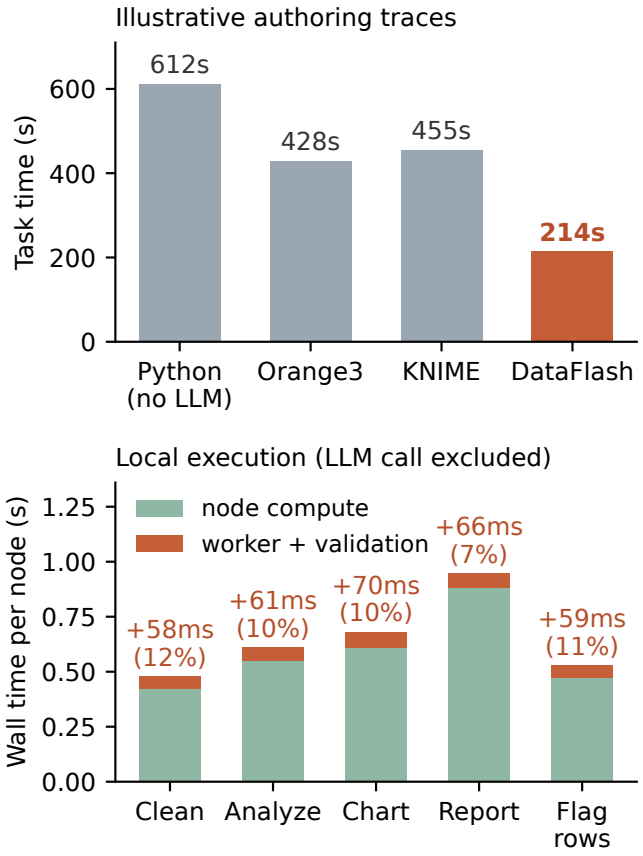



Fig. 3. Illustrative prototype traces, not a controlled user study. (Top) End-to-end authoring time under different interaction regimes. (Bottom) Local execution time excluding the LLM call, with isolation overhead labeled in milliseconds and as a share of each stacked bar.

The timing values in Fig. 3 and Table I are representative prototype traces used to exercise this evaluation design, not a controlled user study. Accordingly, we do not report participant demographics, confidence intervals, or statistical significance. Isolation overhead is the difference between in-process execution and the spawn+validate path over 50 runs per node on a 14-core workstation with 36 GB RAM; it excludes the remote LLM call.

### C. Results

Table I summarizes the illustrative authoring traces. In the illustrative traces, the DataFlash path is 214 s versus 612 s for from-scratch Raw Python. This is not an unbiased estimate of interface productivity: the DataFlash condition receives LLM assistance and the Raw Python condition does not. The result should be read as motivation for the controlled study above, rather than as a causal claim. The reproducibility claim is narrower and artifact-based: a saved `.flow.json` and locked code avoid regeneration. Identical outputs still require unchanged inputs, dependencies, and any relevant random seeds; code locking alone does not establish determinism.

TABLE I
EXPLORATORY AUTHORING TRACES. DIFFERENT ASSISTANCE AND INTERACTION MODES PREVENT CAUSAL COMPARISON OF TOOLS.

| Authoring path | Time (s) |
|---|---|
| Python script, no LLM | 612 |
| Orange3 widget workflow | 428 |
| KNIME widget workflow | 455 |
| DataFlash, LLM-assisted | 214 |

TABLE II
HANDCRAFTED CONTROL CASES FROM THE PROTOTYPE REPORT. RUNTIME PROBES DO NOT ESTABLISH FULL-PATH CONFINEMENT.

| Attempted action | Vector | Control |
|---|---|---|
| Shell command | `os.system(...)` | AST (attr) |
| File read | `open('/etc/passwd')` | AST (call) |
| Reflective escape | `().__class__` | AST (dunder) |
| Pickle RCE | `pd.read_pickle(url)` | AST (attr) |
| Arbitrary import | `import socket` | AST (syntax) |
| Outbound connection | patched `socket` | runtime patch |
| Infinite loop | `while True: pass` | CPU/wall limit |
| Memory bomb | large allocation | RSS watchdog |

Fig. 3 reports 58–70 ms of combined validation and worker-startup overhead across node types, approximately 7–12% of local execution time. This range is across nodes, not a confidence interval. Only aggregate values are available in the paper package; per-run variability and remote generation or repair latency are not available. These numbers therefore cannot quantify the full user-facing execution delay.

Table II maps the original small set of handcrafted cases to individual controls. It is not a systematic attack benchmark. In particular, a direct socket-patch check must be distinguished from the full generated-code path, where importing socket is already rejected. The available record does not document executable harnesses, CPU/RSS/wall thresholds, or whether every runtime probe traversed the full path. The table consequently illustrates control coverage rather than establishing an escape-prevention rate.

**Evaluation limits.** The single case study supports implementation feasibility, not generalization across datasets or tasks. No recorded output assertions establish generation correctness or the benefit of repair. Those claims require new measurements. Likewise, a comparison with equally LLM-assisted Python, matched task outputs, and repeated trials is needed to separate interface effects from model assistance.

## V. RELATED WORK AND CONCLUSION

### A. Related Work

**Visual data-mining tools.** Orange3 [1] and KNIME [2] pioneered widget-based pipelines and remain the reference points for legibility and reproducibility; Node-RED [6] brought the same dataflow metaphor to event-driven systems. DataFlash uses a DAG model and adds intent-driven transformations to built-in and extension nodes. **LLM code generation and agents.** Codex [3] and successors demonstrated program synthesis from natural language, and systems such as SheetCopilot [4] apply it to tabular tasks. These focus on generation quality; DataFlash's contribution is orthogonal—the *defense-in-depth and artifact-locked execution* of such code inside a visual pipeline. **Sandboxing.** RestrictedPython [5] similarly rewrites and restricts the AST, and container/gVisor approaches isolate at the OS layer. DataFlash combines a default-deny AST allowlist with lightweight process isolation, providing statement-level rejection feedback. OS-enforced containment remains necessary for stronger adversarial execution guarantees.

### B. Conclusion and Future Work

DataFlash connects visual workflow authoring, generated Python, AST admission, worker execution, and artifact reuse in one prototype. Its present evidence is limited to a single illustrative workflow and aggregate local overhead values. Application-level checks reduce selected unsafe operations but cannot guarantee filesystem or network confinement.

Future evaluation will measure first-attempt correctness (Pass@1), correctness within two repair attempts (Pass@3), repair outcomes, and failure categories under fixed models, prompts, and decoding settings. Output assertions must be separate from successful execution. Further work includes an equally LLM-assisted Python baseline, repeated timing with dispersion and model-call latency, reproducible full-path security probes with explicit thresholds, and OS-enforced isolation. None of these evaluations is claimed as completed.

## REFERENCES


[1] J. Demšar *et al.*, "Orange: Data Mining Toolbox in Python," *Journal of Machine Learning Research*, vol. 14, pp. 2349–2353, 2013.

[2] M. R. Berthold *et al.*, "KNIME - the Konstanz Information Miner," *ACM SIGKDD Explorations*, vol. 11, no. 1, pp. 26–31, 2009.

[3] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code," *arXiv:2107.03374*, 2021.

[4] H. Li *et al.*, "SheetCopilot: Bringing Software Productivity to the Next Level through Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[5] Zope Foundation, "RestrictedPython: A restricted execution environment for Python," 2023. [Online]. Available: https://github.com/zopefoundation/RestrictedPython

[6] OpenJS Foundation, "Node-RED: Low-code programming for event-driven applications," 2023. [Online]. Available: https://nodered.org